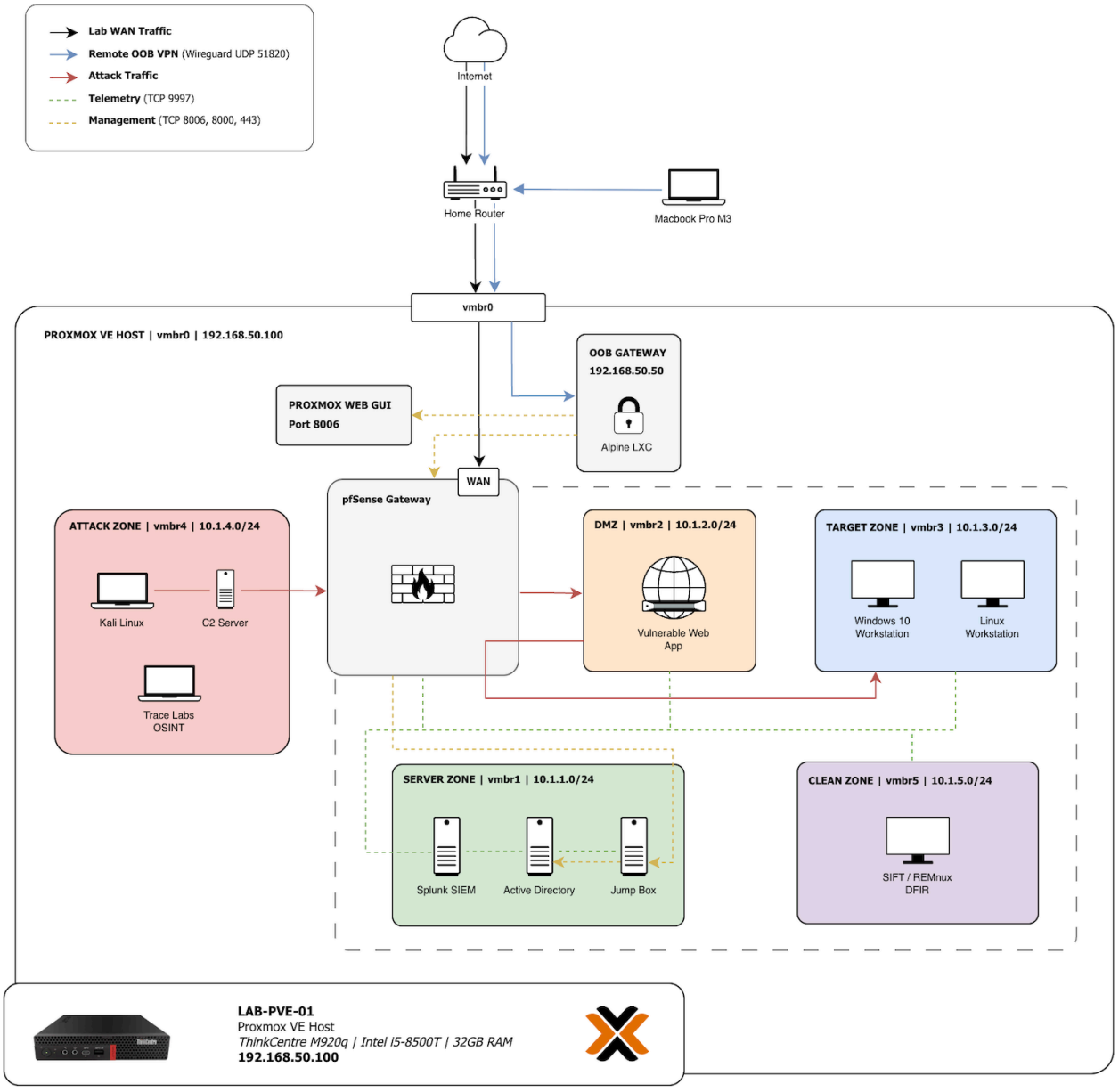
Project 1: Building a Secure Home Cyber Lab: pfSense, Active Directory, and Splunk

System Architecture & End-Goal Topology



Note: This blueprint represents the complete 6-zone enterprise topology strived for across my lab builds. Project 1 establishes the virtual networking, centralized routing, storage policies, and core telemetry ingestion pipeline required to safely support future adversary emulation and analysis phases.

Asset Inventory

Hostname	OS	IP Address	Subnet / Zone	Role
pfSense	FreeBSD	`10.1.x.1`	Gateway Core	Central Router & Policy Firewall
oob-gateway	Alpine Linux	`192.168.50.50`	Out-of-Band	WireGuard LXC remote access

Hostname	OS	IP Address	Subnet / Zone	Role
splunk-siem	Ubuntu Server	`10.1.1.10`	Server Vault (LAN)	Central Log Indexer & Analytics
dc01	Windows Server 2022	`10.1.1.20`	Server Vault (LAN)	Active Directory Domain Controller
win10-jump	Windows 10	`10.1.1.50`	Server Vault (LAN)	Hardened Administration Workstation
win10-client01	Windows 10	`10.1.3.100`	Target Zone (OPT2)	Domain-joined Active Directory client

Network Segmentation Policy

Interface	Subnet	Gateway	DHCP	Outbound Rule Policy
LAN (Server)	`10.1.1.0/24`	`10.1.1.1`	Disabled	Isolated. Custom pass rules for AD/DNS sync only.
OPT1 (DMZ)	`10.1.2.0/24`	`10.1.2.1`	Enabled	Internet allowed; all local lab zones blocked.
OPT2 (Target)	`10.1.3.0/24`	`10.1.3.1`	Enabled	Internet allowed; all local lab zones blocked.
OPT3 (Attack)	`10.1.4.0/24`	`10.1.4.1`	Enabled	Internet allowed; all local lab zones blocked.
OPT4 (Clean)	`10.1.5.0/24`	`10.1.5.1`	Disabled	No internet access allowed.

Hypervisor & Centralized Routing

I set up my lab on a small Lenovo ThinkCentre M920q mini-PC. First, I turned on virtualization in the BIOS, installed Proxmox VE, and swapped the setup over to the free community repository to get rid of the license pop-up.

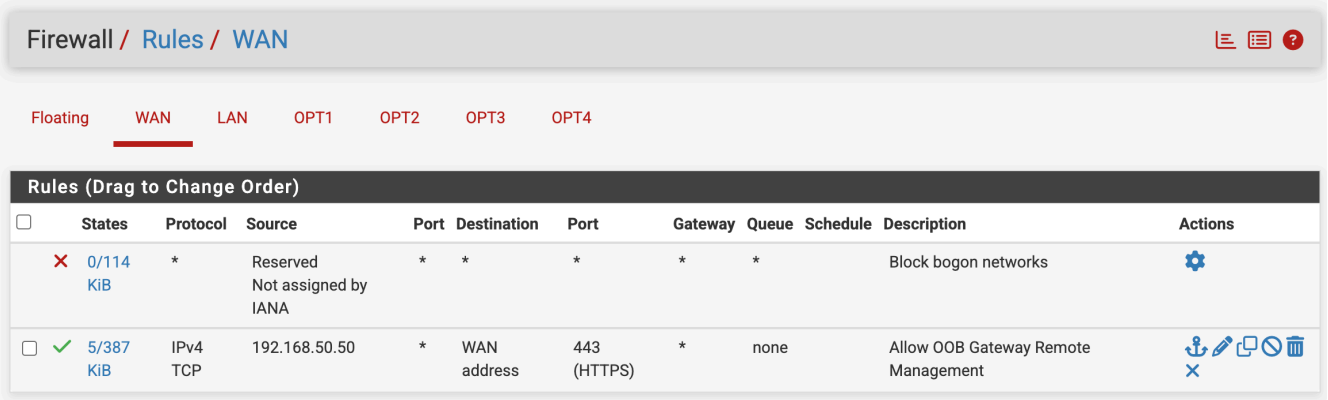
Next, I used Proxmox's virtual switch settings to create five virtual networks named `vmbr1` through `vmbr5`. I left these networks completely disconnected from my physical home network so no traffic could leak out. In the center, I installed pfSense to act as my central router and firewall, mapping my bridge numbers directly to my subnets. I assigned static IPs to my core servers and set up DHCP for the client zones.

To manage the lab remotely without exposing it to the web, I created an unprivileged Alpine Linux container running WireGuard. Because of the safety locks on unprivileged containers, it couldn't talk to

the network cards. I found out I had to manually edit the container's config file on Proxmox to pass through the `/dev/net/tun` device. By keeping the container unprivileged and only passing through this specific device, I ensured that even if an attacker somehow compromises my WireGuard VPN, they are completely trapped inside the container and cannot break out into my physical Proxmox hypervisor.

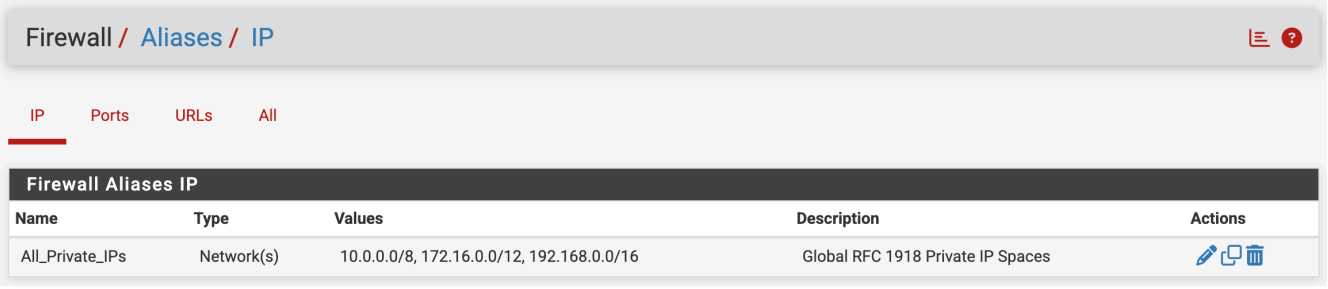
```
root@pve01:~# cat /etc/pve/lxc/101.conf
arch: amd64
cores: 1
features: nesting=1
hostname: OOB-Gateway
memory: 512
net0: name=eth0,bridge=vmbr0,gw=192.168.50.1,hwaddr=BC:24:11:14:E3:C7,ip=192.168.50.50/24,type=veth
onboot: 1
ostype: alpine
rootfs: local-lvm:vm-101-disk-0,size=4G
swap: 512
unprivileged: 1
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net dev/net none bind,create=dir
```

Once that was done, I forwarded port UDP 51820 on my home router, set up a split-tunnel VPN, and turned on NAT routing using iptables inside Alpine. Finally, I went into pfSense and added a rule that only allows HTTPS management access from my WireGuard IP at 192.168.50.50.

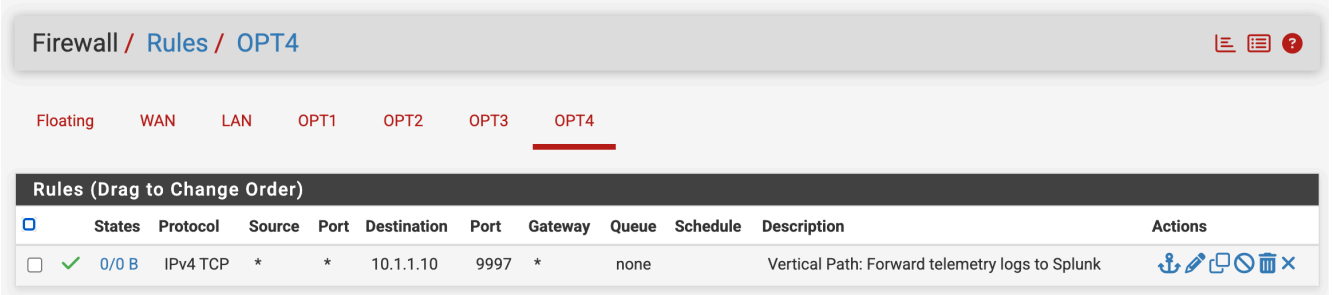


Network Rules & Storage Architecture

To make sure malware couldn't crawl into other networks, I made a pfSense list called `All_Private_IPs` covering my local subnets. On my main networks, I put a simple two-rule security setup at the top of the list: one rule to pass logs up to my Splunk server (10.1.1.10) on port 9997, and another that allows internet access while blocking everything on my local IP list. This let my VMs fetch updates but stopped them from talking to each other.



I set up DHCP on OPT1, OPT2, and OPT3, but kept it off on the server zone (LAN) and the OPT4 malware clean room to keep things stable. On the malware clean room (OPT4) interface, I took away the internet rule completely. Leaving only the Splunk port open meant active malware would stay completely offline but still send logs back to my SIEM.



To get extra space, I plugged in a 1TB Samsung T7 external USB SSD. I formatted it as ext4 because I read it's much better at saving data if the USB cord gets bumped. I grabbed the drive's UUID using `blkid` and added it to my fstab file with the `nofail` flag so Proxmox wouldn't crash on startup if the drive wasn't plugged in.

ID	Type	Content	Path/Target	Shared	Enabled
local	Directory	Backup, Import, ISO image, Container t…	/var/lib/vz	No	Yes
local-lvm	LVM-Thin	Disk image, Container		No	Yes
samsung-t7	Directory	Backup, Disk image, ISO image, Contai…	/mnt/samsung_t7	No	Yes

I split up my storage paths to protect my data: Splunk got a 300GB disk on the external SSD , but I kept pfSense and my Active Directory domain controller on the internal drive. I read that Active Directory databases can corrupt instantly if a USB drive gets disconnected for even a split second. I moved my Windows client and jumpbox to the SSD to free up space.

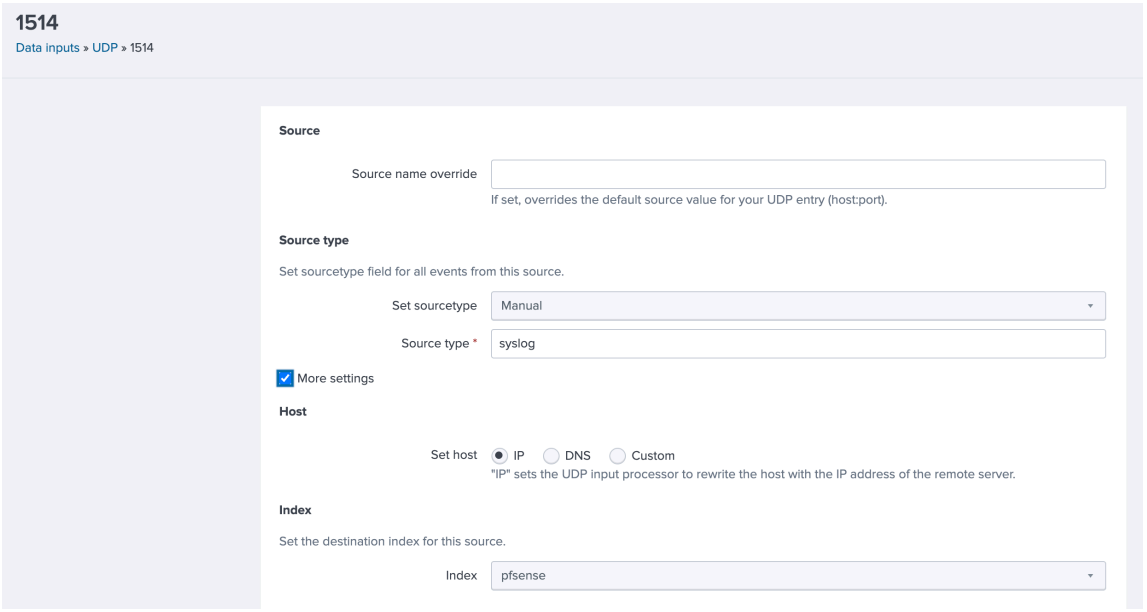
Setting Up Telemetry (Splunk & Syslog)

I set up my main Splunk server inside the server zone. At first, I couldn't reach the internal network from my laptop. I found out the WireGuard container was missing a route back to my internal lab subnets. Adding a static route inside Alpine fixed the issue immediately.

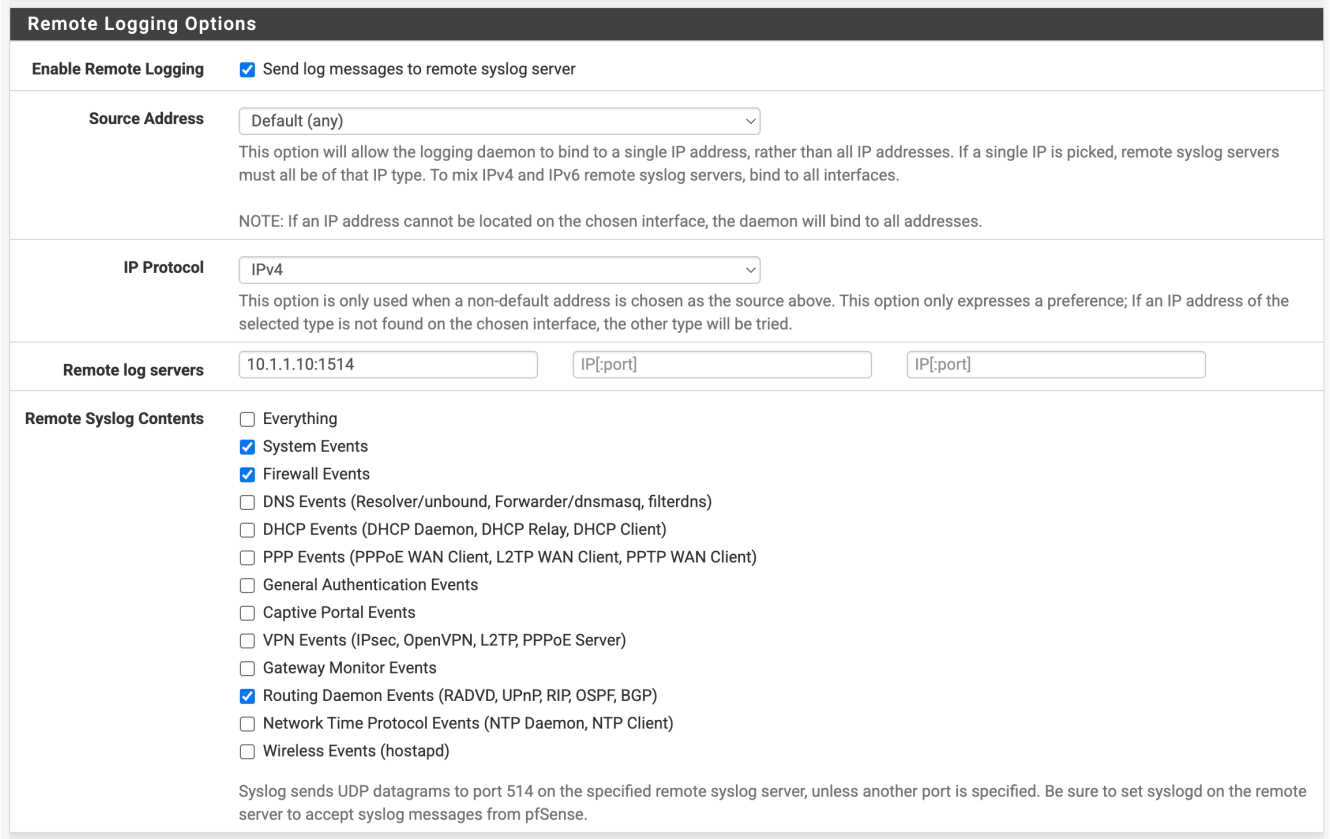
```
OOB-Gateway:~# cat /etc/network/interfaces
auto lo
iface lo inet loopback
iface lo inet6 loopback

auto eth0
iface eth0 inet static
        address 192.168.50.50/24
        gateway 192.168.50.1
        hostname $(hostname)

up ip route add 10.1.0.0/16 via 192.168.50.42
```

I installed Splunk on a fresh Ubuntu server at `10.1.1.10`. Right away, I couldn't log into the console because the Ubuntu installer had defaulted to a US keyboard layout, which scrambled the special characters on my physical Swedish keyboard. I copy-pasted the password over an active SSH session to get in, then fixed the keyboard settings.

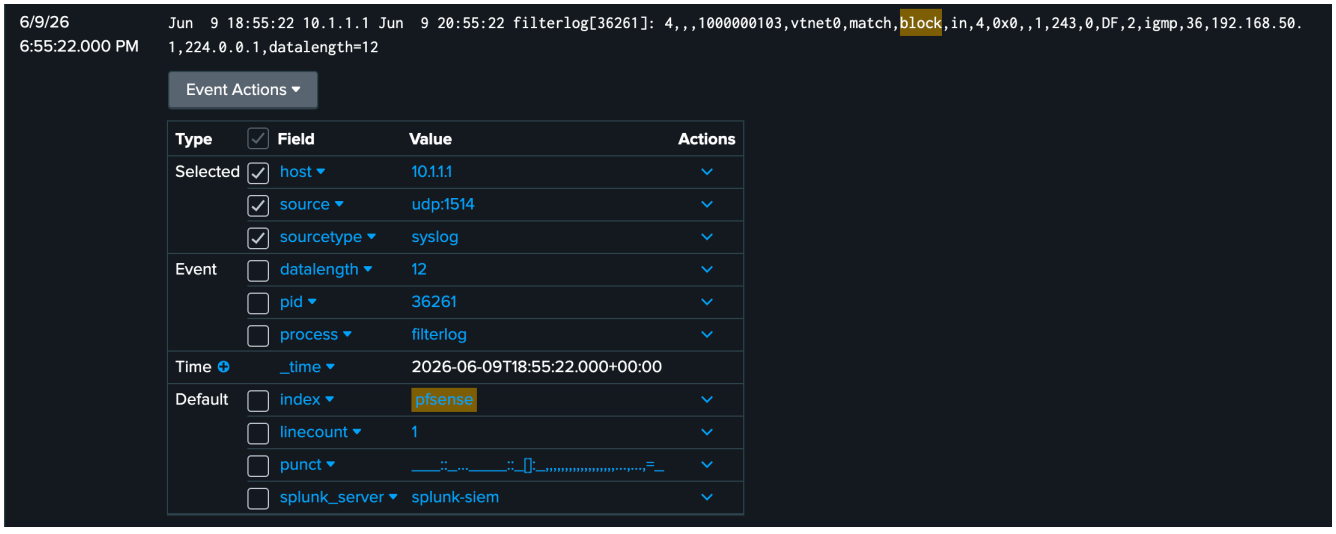
When I tried starting Splunk, it crashed immediately because extracting the files with root privileges left the folders owned by root. A quick `chown` fixed the permissions. I created a firewall index capped at 100 GB and a Linux index capped at 50 GB. I found out that Linux blocks regular accounts from using ports below 1024, so I set up my Splunk listener on port 1514 instead.



In pfSense, I turned on remote logging and pointed it to `10.1.1.10:1514`, which instantly filled my dashboard with logs.



To collect operating system logs from my Linux client, I installed the Splunk forwarder, but it wouldn't read the command history file. I realized my sudo command had left the file owned by root. After changing file ownership and granting the forwarder permissions to read the home directory, the logs started populating.



To send logs from my Windows target (`win10-client01`) to Splunk, I installed the Splunk Universal Forwarder. However, default Windows logs are incredibly noisy and hard to use for tracking threats.

To get cleaner data, I installed Microsoft Sysmon and applied Olaf Hartong's configuration. This acts like a filter that ignores normal, boring background activity and only logs high-value security events like programs launching or network connections.

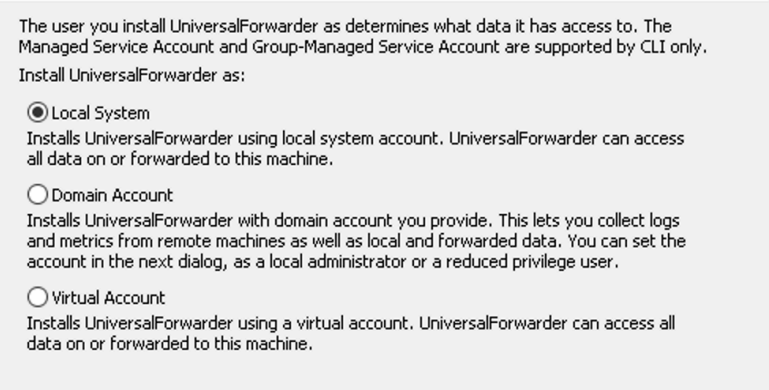
Finally, I opened the Splunk Forwarder's `inputs.conf` file and added the paths for these new Sysmon logs. After restarting the forwarder, I hopped over to Splunk and verified that basic Windows and Sysmon events were flowing perfectly into my `windows` index.

inputs - Notepad

File Edit Format View Help

```
[WinEventLog://Application]
index = windows
disabled = false

[WinEventLog://Security]
index = windows
disabled = false

[WinEventLog://System]
index = windows
disabled = false

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = windows
disabled = false
renderXml = true
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational
```

Next, I set up telemetry on my domain controller (dc01) to catch active directory attacks. I installed Sysmon64 using Olaf Hartong's configuration file to track process creation and network traffic. When I tried running the Splunk forwarder installer, it kept rolling back and crashing. I found out that because a domain controller has no local user database, the default "Virtual Account" installer setting fails when it tries to add itself to local performance monitoring groups. Changing the installation service account to "Local System" resolved the issue instantly, allowing the install to finish and granting the forwarder the elevated permissions it needed to read restricted directory logs.



Once the logs started flowing, I noticed the domain controller was filling my SIEM with a lot of background noise. It generated so many events from Windows Filtering Platform connections and file handles that I was worried I would run out of space on my free 500 MB daily license limit.

To fix this, I opened C:\Program Files\SplunkUniversalForwarder\etc\system\local\inputs.conf on the DC and added the line `blacklist1 = EventCode="(5156|5158|4656|4658|4690|4907)"` to drop repetitive, useless background logs.

```
[WinEventLog://Security]
index = windows
disabled = false
blacklist1 = EventCode="(5156|5158|4656|4658|4690|4907)"
```

I restarted SplunkForwarder to apply the changes, which reduced my DC's log volume significantly while keeping critical logon and policy modifications intact.

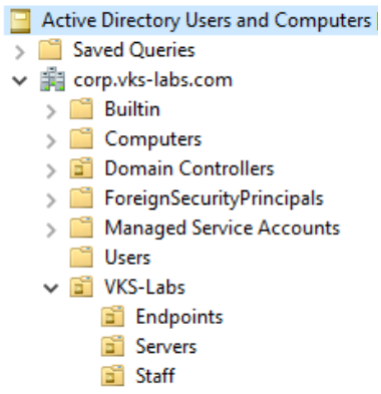
Active Directory & Domain Policy

I built a Windows Server 2022 domain controller at 10.1.1.20. I used standard drive settings so Windows could read the disk, mapped the network card to the VirtIO driver, and turned off the hypervisor firewall so pfSense would handle the security. Since Windows doesn't have VirtIO drivers

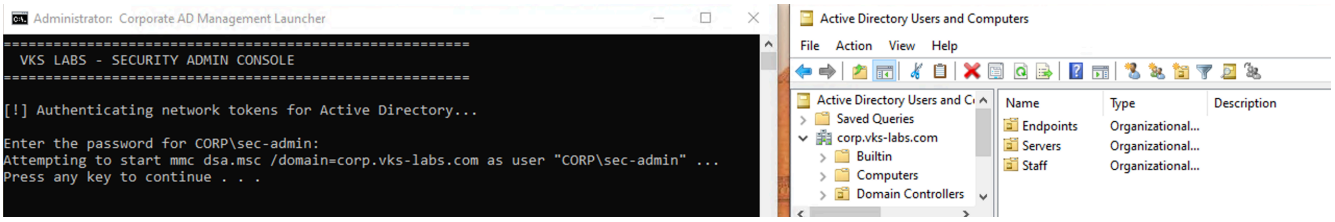
built-in, it booted completely offline. I mounted the guest driver ISO to the virtual CD drive and used Device Manager to install the network card.

```
CD/DVD Drive (ide1)          local:iso/virtio-win.iso,media=cdrom,size=771138K
```

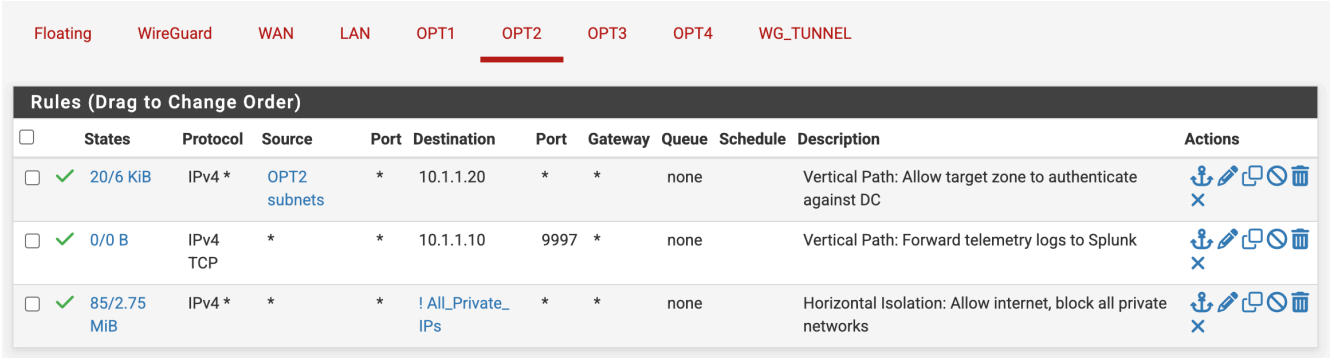
I changed the server name to `dc01` and promoted it to a domain controller for `corp.vks-labs.com`. I also set up a folder structure inside AD with Endpoints, Servers, and Staff folders.



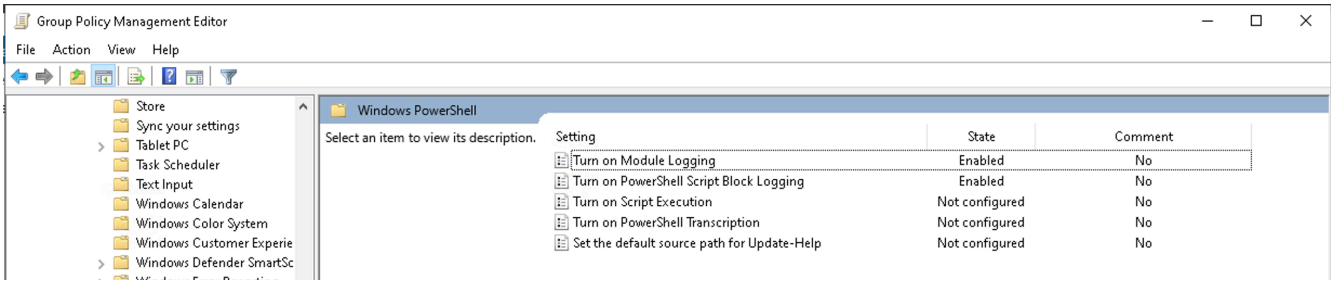
I configured a Windows Jump Box at 10.1.1.50 but left it unjoined from the domain. This ensures Domain Admin passwords are never saved in its memory where an attacker's tool could find them. I used remote management tools and a custom script to securely manage AD from this unjoined workstation, using the DC's console only for Group Policy edits.



When I tried to join the Windows client in OPT2 to the domain, the connection timed out. I traced this to my firewall isolation rule blocking local traffic. Dragging an AD authentication pass rule to the top of my OPT2 rule list fixed the timeout instantly.



To capture command-line activity, I made a domain Group Policy and enabled PowerShell script block logging and module logging.



The client initially failed to fetch this policy. I found out there was a 9-hour time drift between my host's motherboard clock and the DC. Because Active Directory rejects any clock difference over 5 minutes, I forced the client to sync with the DC using the `net time` command, then fixed the server's timezone.

Even with the time aligned, test commands wouldn't show up in Splunk. A policy report showed that the client computer object was sitting in the default `Computers` folder, which doesn't support policy inheritance. I dragged the machine object into my `Endpoints` folder, renamed the computer to `win10-client01`, and rebooted.

```
PS C:\Windows\system32> net time \\10.1.1.20 /set /y
Current time at \\10.1.1.20 is 2026-07-06 19:03:06

The command completed successfully.

PS C:\Windows\system32>
PS C:\Windows\system32> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.
```

After rebooting, the GPO applied perfectly, but logs still weren't populating Splunk because the forwarder's configurations lacked a path for the PowerShell log channel. I opened `inputs.conf` in Notepad, manually appended the path, and restarted the forwarder.

inputs - Notepad

File Edit Format View Help

```
[WinEventLog://Application]
index = windows
disabled = false

[WinEventLog://Security]
index = windows
disabled = false

[WinEventLog://System]
index = windows
disabled = false

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = windows
disabled = false
renderXml = true
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational

[WinEventLog://Microsoft-Windows-PowerShell/Operational]
index = windows
disabled = false
renderXml = true
```

I opened a shell on the client, typed a test command, and watched it immediately register as EventCode 4104 in Splunk!